

Explain DP Approaches

Yes. The file is explaining a **DP pattern** based on “current state + possible previous states”, where `ways` represents the possible transitions. I'll explain it from the problem intuition all the way to **memoization** → **tabulation** → **space optimization**, and I'll also make clear **why 1D DP is enough instead of 2D**.

1. First understand what kind of question this pattern solves

The general problem is:

You have a target/state, and from a state you can make several possible moves. Find the minimum/maximum/best answer needed to reach the target.

For example:

```
ways = {1, 2, 3}
target = 5
```

This means that to reach 5, your last move could have been:

```
4 → 5    using way = 1
3 → 5    using way = 2
2 → 5    using way = 3
```

So the important observation is:

To calculate the answer for state `i`, look at all states from which I could have reached `i`.

The file describes exactly this recurrence:

```
dp[i] = min(dp[i-1], dp[i-2], dp[i-3]) + cost[i];
```

when the available moves are `{1,2,3}`.

2. What exactly does `dp[i]` mean?

This is the **most important part of DP**.

We define:

```
dp[i] = minimum cost required to reach state i
```

So:

```
dp[0] → minimum cost to reach 0
dp[1] → minimum cost to reach 1
dp[2] → minimum cost to reach 2
dp[3] → minimum cost to reach 3
...
dp[5] → minimum cost to reach 5
```

The file emphasizes that the meaning of `dp[i]` must be defined before writing the transition.

And notice:

`ways` **does NOT become a DP dimension**.

If:

```
ways = {1,2,3}
```

then `ways` simply tells us:

Which previous states should I try?

It doesn't represent information that we need to remember.

That's why:

```
dp[i]
```

is enough instead of:

```
dp[i][j]
```

We'll come back to this because it's extremely important.

3. Deriving the recurrence

Suppose:

```
target = 5
ways = {1,2,3}
```

We're currently calculating:

```
dp[5]
```

Ask:

```
"What could my last move have been?"
```

There are three possibilities.

Last move = 1

We came from:

```
4 → 5
```

So candidate:

```
dp[4] + cost
```

Last move = 2

We came from:

```
3 → 5
```

Candidate:

```
dp[3] + cost
```

Last move = 3

We came from:

```
2 → 5
```

Candidate:

```
dp[2] + cost
```

Therefore:

```
dp[5] = min(
    dp[4],
    dp[3],
    dp[2]
) + cost;
```

Generalizing:

```
dp[i] = min(dp[i - way] + cost)
```

for every valid way.

That's the entire DP recurrence.

4. Why don't we calculate the entire path?

This is the power of DP.

Suppose we're calculating:

```
dp[5]
```

We don't need to enumerate every complete path:

```
0 → 1 → 2 → 3 → 4 → 5
0 → 1 → 3 → 5
0 → 2 → 5
0 → 3 → 5
...
```

Instead, we trust that:

```
dp[2]
dp[3]
dp[4]
```

already contain their optimal answers.

Then:

```
dp[5]
```

only needs to ask:

Which of these previous states gives me the cheapest way to reach 5?

This is the principle in the file: store the best answer for each previous state rather than recomputing complete paths.

5. Now let's solve it using Memoization

Memoization is **Top-Down DP**.

The thought process is:

"I want the answer for `target`. I'll recursively ask for smaller states."

Suppose:

```
target = 4
ways = {1, 2}
```

We start:

```
solve(4)
```

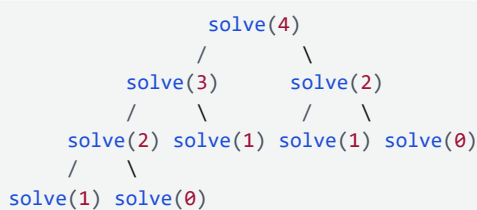
What can be the last move?

```
4 ← 3    using 1
4 ← 2    using 2
```

Therefore:

```
solve(4)
=
min(
  solve(3),
  solve(2)
) + 1;
```

The recursive tree becomes:



The problem is obvious:

```
solve(2)
```

is calculated multiple times.

The file demonstrates exactly this overlapping-subproblem problem.

6. Memoization fixes the repeated work

We create:

```
memo[target]
```

which stores:

"Have I already calculated the answer for this state?"

Initially:

```
memo[i] = -1;
```

Then:

```
if (memo[target] != -1)
    return memo[target];
```

means:

```
"If I have already solved this state, don't solve it again. Just return the stored answer."
```

And after calculating:

```
return memo[target] = result;
```

we save it.

The process becomes:

```
solve(2)
  ↓
calculate
  ↓
memo[2] = answer
```

Later:

```
solve(2)
  ↓
memo[2] already exists
  ↓
return memo[2]
```

So every state is essentially solved once.

7. Memoization code

For the simple example where every move costs 1:

```
vector<int> memo;

int solve(int target, vector<int>& ways) {
    if (target == 0)
        return 0;

    if (memo[target] != -1)
        return memo[target];

    int result = INT_MAX;

    for (int way : ways) {
        if (way <= target) {
            result = min(result, solve(target - way, ways) + 1);
        }
    }

    return memo[target] = result;
}
```

Then:

```
memo.assign(target + 1, -1);
int answer = solve(target, ways);
```

What happens for target = 5?

```
solve(5)
|
+--- solve(4)
|   |
|   +--- solve(3)
|   +--- solve(2)
|
+--- solve(3)
|
+--- solve(2)
```

But after `solve(3)` or `solve(2)` has been calculated once, the memo prevents recalculation.

8. Why is the base case `target == 0`?

We say:

```
if (target == 0)
    return 0;
```

Because:

The cost of reaching the starting state is zero.

Imagine:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

To calculate:

`dp[1]`

using move 1:

`dp[1] = dp[0] + 1;`

Therefore we need:

`dp[0] = 0`

giving:

`dp[1] = 0 + 1 = 1`

This is why 0 acts as the foundation of the DP.

9. Now convert Memoization → Tabulation

This is where you need to understand the **direction of computation**.

Memoization

We start from the answer:

```
solve(5)
  ↓
solve(4), solve(3), solve(2)
  ↓
smaller states
  ↓
solve(0)
```

So it goes:

TARGET → smaller → base

Tabulation

We reverse the direction:

base → smaller → larger → target

So:

```
dp[0]
  ↓
dp[1]
  ↓
dp[2]
  ↓
dp[3]
  ↓
dp[4]
  ↓
dp[5]
```

The file explicitly describes bottom-up as calculating smaller states first and then using them to calculate larger states.

10. Tabulation initialization

We create:

```
vector<int> dp(target + 1, INT_MAX);
```

So initially:

```
dp[0] = 0
dp[1] = ∞
dp[2] = ∞
dp[3] = ∞
dp[4] = ∞
dp[5] = ∞
```

Then:

```
dp[0] = 0;
```

Why INT_MAX?

Because we're finding a **minimum**.

We start with:

```
∞
```

and every valid answer will be smaller.

For example:

```
dp[5] = ∞

candidate = 7

dp[5] = min(∞, 7)
        = 7
```

Then:

```
candidate = 6

dp[5] = min(7, 6)
        = 6
```

The file explains this initialization logic in detail.

11. Tabulation code

```
vector<int> dp(target + 1, INT_MAX);
dp[0] = 0;

for (int i = 1; i <= target; i++) {
    for (int way : ways) {
        if (way <= i) {
            dp[i] = min(dp[i], dp[i - way] + 1);
        }
    }
}

return dp[target];
```

Let's understand this line by line.

Outer loop

```
for (int i = 1; i <= target; i++)
```

means:

```
"I'm going to calculate the answer for every state."
```

So:

```
i = 1 → calculate dp[1]
i = 2 → calculate dp[2]
i = 3 → calculate dp[3]
i = 4 → calculate dp[4]
i = 5 → calculate dp[5]
```

Inner loop

```
for (int way : ways)
```

means:

"For the current state, try every possible transition."

For:

```
ways = {1,2,3}
```

when:

```
i = 5
```

we try:

```
way = 1
way = 2
way = 3
```

The file describes this as trying every possible way to arrive at the current state.

12. The most important line

```
dp[i] = min(dp[i], dp[i - way] + 1);
```

Suppose:

```
i = 5
way = 2
```

Then:

```
i - way
```

becomes:

```
5 - 2 = 3
```

So:

```
dp[5] = min(dp[5], dp[3] + 1);
```

Meaning:

"If I can reach 3 optimally, then I can go from 3 to 5 using a move of 2."

So:

```
cost to reach 5
=
cost to reach 3
+
cost of current move
```

This is the exact transition explained in the source.

13. Why `if (way <= i)`?

Suppose:

```
i = 2
way = 3
```

Then:

```
i - way = -1
```

We would be asking for:

```
dp[-1]
```

which is invalid.

Therefore:

```
if (way <= i)
```

means:

Only use a transition if that transition is possible from the current state.

14. Complete tabulation example

Take:

```
ways = {1,2,3}  
target = 5
```

and every move costs 1.

Initial:

```
dp = [0, ∞, ∞, ∞, ∞, ∞]
```

Calculate $dp[1]$

Only way 1 works:

```
dp[0] + 1 = 1
```

So:

```
dp[1] = 1
```

Now:

```
[0, 1, ∞, ∞, ∞, ∞]
```

Calculate $dp[2]$

Using 1:

```
dp[1] + 1 = 2
```

Using 2:

```
dp[0] + 1 = 1
```

Therefore:

```
dp[2] = 1
```

Array:

```
[0, 1, 1, ∞, ∞, ∞]
```

Calculate $dp[3]$

```
dp[2] + 1 = 2
```

```
dp[1] + 1 = 2
```

```
dp[0] + 1 = 1
```

Therefore:

```
dp[3] = 1
```

Calculate $dp[4]$

```
dp[3] + 1 = 2
```

```
dp[2] + 1 = 2
```

```
dp[1] + 1 = 2
```

Therefore:

```
dp[4] = 2
```

Calculate `dp[5]`

```
dp[4] + 1 = 3
dp[3] + 1 = 2
dp[2] + 1 = 2
```

Therefore:

```
dp[5] = 2
```

Final:

```
dp = [0,1,1,1,2,2]
```

Answer:

```
2
```

One optimal path:

```
0 → 2 → 5
```

Exactly as demonstrated in the source.

15. Now the important part: Space Optimization

Our tabulation uses:

```
dp[0]
dp[1]
dp[2]
...
dp[target]
```

So space is:

```
O(target)
```

But ask:

Do we really need the entire `dp` array?

For this particular recurrence, the answer can sometimes be **no**.

If the transition is something like:

```
dp[i] = min(dp[i-1], dp[i-2], dp[i-3]) + cost;
```

then to calculate `dp[i]`, we only need:

```
dp[i-1]
dp[i-2]
dp[i-3]
```

We don't need:

```
dp[i-100]
```

anymore.

So we can maintain only the required previous states.

16. Example of space optimization

Suppose:

```
ways = {1,2,3}
```

Then:

```
dp[i] depends on:
dp[i-1]
dp[i-2]
dp[i-3]
```

So instead of:

```
vector<int> dp(target + 1);
```

we could maintain:

```
a = dp[i-3]
b = dp[i-2]
c = dp[i-1]
```

Then:

```
current = min({a, b, c}) + cost;
```

After calculating it, shift:

```
a ← b
b ← c
c ← current
```

This gives:

```
int a = dp[0];
int b = ...;
int c = ...;

for (...) {
    int current = min({a, b, c}) + cost;

    a = b;
    b = c;
    c = current;
}
```

The exact initialization depends on the **base cases and cost definition of the actual problem**. You should not blindly apply this optimization to every DP.

17. But here's an important correction to the idea of "space optimization"

Don't assume:

"Every 1D DP can be reduced to $O(1)$."

That's false.

The key question is:

How many previous states does my transition actually need?

For example:

```
dp[i] = dp[i-1] + dp[i-2]
```

Only two previous states are required:

```
prev2
prev1
current
```

So:

```
 $O(n) \rightarrow O(1)$ 
```

is possible.

But if:

```
dp[i] = min(dp[i-1], dp[i-2], ..., dp[i-k])
```

and k is large, you may need to maintain many previous values, or use another data structure such as a deque depending on the exact recurrence.

18. Now your biggest confusion: Why not `dp[i][j]`?

This is the part you should **really understand**, because it will prevent you from blindly creating 2D DP.

Suppose:

```
ways = {1, 2, 3}
```

You might think:

```
dp[i][j]
```

because there are:

```
i = target  
j = way
```

But this is wrong reasoning.

A variable being present in a loop does **not** automatically mean it deserves a DP dimension.

The source makes this distinction explicitly.

Think about what `j` means

Here:

```
for (int j = 0; j < ways.size(); j++)
```

`j` simply means:

```
"Which possible move am I trying right now?"
```

For:

```
ways = {1,2,3}
```

we try:

```
j = 0 → way = 1  
j = 1 → way = 2  
j = 2 → way = 3
```

Once we've tried all three, we don't need to remember:

```
"I was previously checking way 2."
```

That information has no effect on future states.

Therefore:

```
dp[i]
```

is sufficient.

19. When WOULD you need 2D?

Suppose the question changes to:

```
"Find the answer using only the first j types of coins."
```

Now `j` becomes important.

Because:

```
dp[1][10]
```

could mean:

```
Make 10 using only coin type 1.
```

while:

```
dp[3][10]
```

means:

```
Make 10 using coin types 1, 2 and 3.
```

Now changing `j` changes the **available choices**.

Therefore:

```
dp[j][amount]
```

is a legitimate 2D state.

The source uses this exact distinction with coin-change and 2D state reasoning.

20. The easiest test for 1D vs 2D

Whenever you are confused, ask:

"If I remove this variable from my state, do I lose information needed to solve future states?"

```
ways[j]
```

Remove `j`:

No problem.

Because `j` was only selecting a transition.

Therefore:

```
dp[i]
```

Knapsack item index

Remove item index:

Problem changes.

Because you need to know which items are available.

Therefore:

```
dp[i][capacity]
```

This is the exact mental test described in the source.

21. The complete transformation

You should see the three versions as **the same recurrence**, not three different algorithms.

Step 1 — Recursion

```
solve(i)
```

Means:

Find the answer for state `i`.

Step 2 — Memoization

```
memo[i]
```

means:

Store the answer of `solve(i)` so we don't calculate it again.

Structure:

```
int solve(int i) {
    if (i == 0)
        return 0;

    if (memo[i] != -1)
        return memo[i];

    int ans = INT_MAX;

    for (int way : ways) {
        if (way <= i)
            ans = min(ans, solve(i - way) + cost);
    }

    return memo[i] = ans;
}
```

Step 3 — Tabulation

Remove recursion and calculate states in dependency order:

```
vector<int> dp(target + 1, INT_MAX);
dp[0] = 0;

for (int i = 1; i <= target; i++) {
    for (int way : ways) {
        if (way <= i) {
            dp[i] = min(dp[i], dp[i - way] + cost);
        }
    }
}

return dp[target];
```

Step 4 — Space optimization

Look at the recurrence:

```
dp[i] = min(previous required states) + cost;
```

Ask:

"How many old states do I actually need?"

If only a fixed number are required, maintain only those values instead of the entire array.

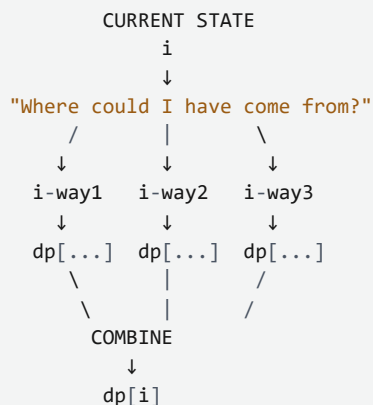
22. The DP pattern you should actually memorize

Don't memorize this:

```
for (int i = 1; i <= target; i++)
    for (int way : ways)
```

That's just implementation.

Memorize this thought process:



And ask these 4 questions every time:

1. What is my state?

```
dp[i] = ?
```

2. What previous states can reach it?

```
i - way
```

3. What do I get from that previous state?

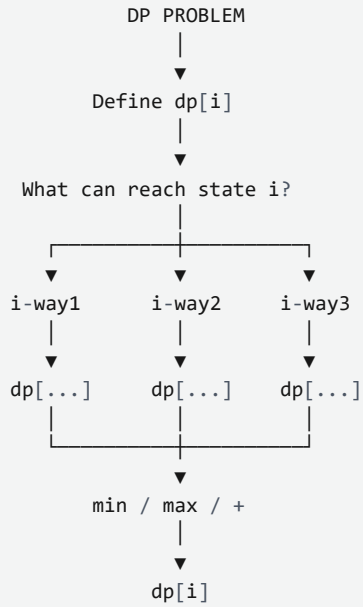
```
dp[i-way] + current cost/value
```

4. How do I combine the possibilities?

```
min
max
+
OR
AND
```

The source ends with essentially this four-question framework.

Final picture



Then:

Recursive
↓
Memoization
↓
Tabulation
↓
Space Optimization

The **recurrence does not fundamentally change**. What changes is **how we store and compute the states**.

And the most important distinction from this entire pattern is:

`ways` tells you how to move between states; it is not automatically a state dimension.
`dp[i][j]` is justified only when `j` contains information that changes what answers are possible in the future.